

Baranya vármegye EJSZ Széchenyi István gimnázium és technikum

Vizsgaremek

Készítette:

Bräutigam Richárd

Erdei Tamás

Husi Krisztóf

Pécs
2025

Baranya vármegye EJSZ Széchenyi István gimnázium és technikum

Szakma megnevezése: Szoftverfejlesztő és tesztelő A

szakma azonosító száma: 506131203

Vizsgaremek

Bookify

Készítették:

Bräutigam Richárd

Erdei Tamás

Husi Kristóf

Feladat rövid ismertetése	4
Bevezetés	4
Cél	4
Jövőbeli tervek	4
Használt technológiák	4
Programozási nyelvek	4
Adatbázis típusa	4
Egyéb technológiák	5

Adatbázis	5
Adatbázis rövid magyarázata	5
Fejlesztés módszertan	7
Scrum	7
Elvégzett tesztek	8
Tesztelési fajták	8
Jelentőség	8
Használt technológiák	9
Program működésének a részletes leírása	9
Program részletes leírása	9
Fejlesztői dokumentáció	11
Fejlesztési technológiák	11
HTTP protokollok	11
A rétegek szerepköre a projektben	12
Frontend részletes leírása	12
Frontend egyes részein kiemelkedő megoldások	14
Lokális szerver futtatása	15
Csapatmunka	16
Szerepek	16
Jövőbeli terveink	16

Feladat rövid ismertetése

Bevezetés

Az online időpontfoglaló rendszer célja, hogy digitálisan lehetővé tegye az időpontfoglalást különböző szolgáltatásokra, például fodrászat, orvosi rendelés, tanácsadás stb. A rendszer elérhető böngészőből, platformfüggetlenül, és biztosítja a gyors, egyszerű és biztonságos időpontfoglalást mind a felhasználók, mind a szolgáltatók számára.

Cél

A projekt célja egy olyan webes alkalmazás létrehozása, amely felhasználóbarát felületet biztosít az ügyfelek számára időpont foglalására, valamint egy adminisztrációs panelt a szolgáltatók részére az időpontok kezeléséhez. A rendszer fejlesztése során fontos szempont volt a stabil működés, egyszerű használhatóság, valamint a reszponzivitás.

Jövőbeli tervek

A rendszer jövőbeli bővítései között szerepel a következők implementálása:

- Email-értesítések és emlékeztetők bevezetése
- Naptár szinkronizáció (pl. Google Calendar)
- Mobilalkalmazás fejlesztése
- Online fizetés integrálása (pl. Stripe, PayPal)

Használt technológiák

Programozási nyelvek

A rendszer fejlesztéséhez az alábbi nyelveket használtuk:

- HTML5 és CSS3: a felhasználói felület kialakításához
- JavaScript: a kliensoldali interaktivitáshoz
- PHP: a szerveroldali logikához

Adatbázis típusa

Az alkalmazás adatainak tárolására MySQL-t alkalmaztunk, amely egy relációs adatbázis-kezelő rendszer. Előnye, hogy strukturált adatkezelést biztosít, és széles körben támogatott.

Egyéb technológiák

1. Jira

A Jira-t a projekt menedzsmentjére és a feladatok nyomon követésére használtuk.

Segített abban, hogy:

- strukturáltan kezeljük a feladatokat (pl. backlog, sprint board),
- átlátható legyen a csapaton belüli munkaelosztás,
- hatékonyabban követhessük nyomon a fejlesztési folyamat állapotát (pl. státusz: „To Do”, „In Progress”, „Done”).

2. Figma

A Figma-t használtuk a felhasználói felület megtervezéséhez. Előnyei:

- valós idejű kollaborációt tesz lehetővé a csapat tagjai között,
- platformfüggetlen (webalapú), nem kell telepíteni,
- modern, intuitív felületet biztosít a drótvázak és prototípusok készítéséhez.

3. Git

A Git egy elengedhetetlen eszköz a modern szoftverfejlesztésben. Azért választottuk, mert:

- biztosítja a forráskód verziókezelését és visszagörgethetőségét,
- lehetővé teszi a párhuzamos fejlesztést több ágon (branch-ek),
- egyszerűen integrálható platformokkal (pl. GitHub, GitLab, Bitbucket).

Adatbázis

Adatbázis rövid magyarázata

Az adatbázis relációs struktúrával lett kialakítva MySQL-ben, és hat fő táblát tartalmaz:

1. users (felhasználók):

Tartalmazza az összes regisztrált felhasználó adatait. Fő mezők:

- `id`: azonosító
- `name`, `email`, `password`, `phone`: a felhasználó elérhetőségi adatai

- `role`: meghatározza, hogy a felhasználó egyszerű felhasználó, szolgáltató vagy admin
- `register_date`: regisztráció időpontja

2. providers (szolgáltatók):

Ez a tábla a szolgáltatói adatokat tárolja. A `user_id` mező hivatkozik a `users` táblára. Mezők:

- `type`, `description`, `name`: szolgáltatás típusa, leírása, neve
- `working_hours`, `address`, `phone_number`: elérhetőségi és nyitvatartási adatok
- `price`, `duration`: szolgáltatás ára és időtartama

3. appointments (időpontfoglalások):

Ide kerülnek az időpont foglalások. Hivatkozik a `user_id` és `provider_id` mezőkön keresztül a `users` és `providers` táblára.

- `appointment_date`: a lefoglalt időpont
- `status`: az időpont állapota (`pending`, `confirmed`, `completed`, `canceled`)

4. ratings (értékelések):

A felhasználók által adott értékelések. Az `appointment_id` segítségével kapcsolódik a konkrét foglaláshoz.

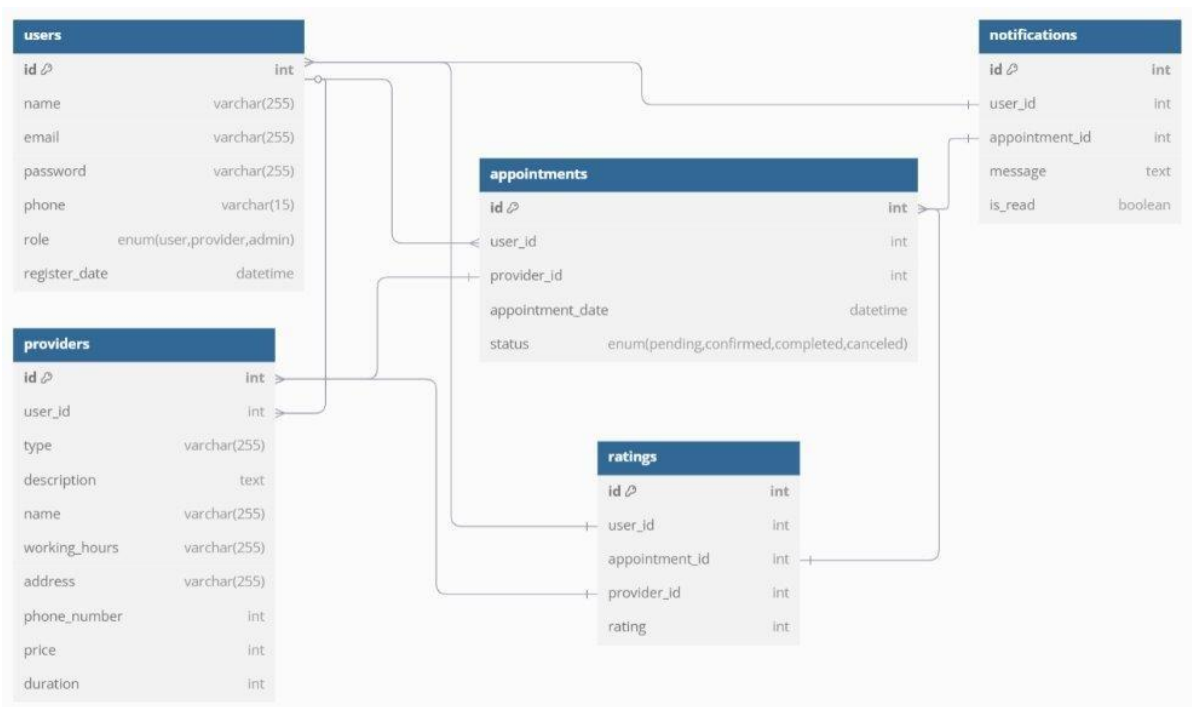
- `rating`: számérték (pl. 1-5), amely a szolgáltatás minőségét mutatja

5. notifications (értesítések):

Az alkalmazás automatikus vagy kézi üzeneteit tartalmazza.

- `user_id`: a címzett felhasználó
- `appointment_id`: mely foglaláshoz kapcsolódik
- `message`: maga az üzenet
- `is_read`: boolean típus, hogy a felhasználó elolvasta-e

Ez a logikus és jól normalizált adatmodell lehetővé teszi az adatok közötti kapcsolat könnyű kezelését, és biztosítja a rendszer átláthatóságát és bővíthetőségét.



Fejlesztés módszertan

Scrum

- A fejlesztés során a **Scrum** agilis módszertant alkalmaztuk, amely lehetővé teszi a fokozatos, iteratív fejlesztést és a folyamatos visszacsatolást. A Scrum előnye, hogy rugalmasan reagál a változó igényekre és elősegíti a csapat együttműködését.

Szerepekörök:

- **Scrum Master:** Felelős volt a folyamatok betartásáért és a csapat munkájának támogatásáért.
- **Product Owner:** Meghatározta a projekt célját, és priorizálta a feladatokat.
- **Fejlesztői csapat:** Részt vettek a feladatok kivitelezésében, kódolásban, tesztelésben.

Folyamat:

- **Sprint tervezés:** A projektet 3 hetes sprintekre osztottuk. Minden sprint elején meghatároztuk a célokat és kiválasztottuk a megvalósítandó feladatokat a backlogból.
- **Napi stand-up:** Rövid, napi megbeszéléseken (15 perc) mindenki beszámolt arról, hogy min dolgozik, mi készült el, és mi akadályozhatja a haladást.

- **Fejlesztés és tesztelés:** A feladatokat a sprint során folyamatosan fejlesztettük és teszteltük.
- **Sprint review (értékelés):** Minden sprint végén bemutattuk a működő funkciókat, és értékeltük az elért eredményeket.
- **Retrospective:** Átbeszéltük, mi ment jól, és miben tudunk fejlődni a következő sprintben.

Használt eszközök:

- **Jira:** Feladatkezeléshez és sprint backlog vezetéséhez
- **GitHub:** Verziókezeléshez és együttműködéshez

Elvégzett tesztek

Tesztelési fajták

Tesztelés során két alapvető típust különböztettünk meg. Manuális és unit tesztelést. Mindkettőnek saját jelentősége és szerepe van.

1. Manuális tesztelés

- A manuális tesztelés során a szoftvert kézzel vizsgálják meg annak érdekében, hogy kiderüljön, megfelelően működik-e az elvárásoknak megfelelően. A tesztelő különböző funkciókat próbál ki, figyeli a rendszer viselkedését, és jelzi, ha hibát tapasztal. Ezt a módszert főként a felhasználói felület és az alkalmazás funkcióinak ellenőrzésére alkalmazzák.

2. Unit tesztelés

- Az unit tesztelés célja, hogy a program legkisebb egységeit – például függvényeket vagy osztályokat – külön-külön ellenőrizzük. A tesztelés általában automatikusan történik speciális keretrendszerek segítségével, és segít abban, hogy a fejlesztés során a kód stabil és hibamentes maradjon.

Jelentőség

1. Manuális tesztelés

- A manuális tesztelés lehetővé teszi, hogy a programot emberi szemmel, valós felhasználói szemszögből vizsgáljuk meg. Ennek során olyan hibák is észrevehetők, amelyeket automatizált tesztek nem feltétlenül fednek fel, például vizuális eltérések, logikai hibák vagy felhasználói élménnyel kapcsolatos problémák. A manuális tesztelés különösen fontos a végfelhasználói funkciók, interakciók és a

rendszer általános működésének értékeléséhez. Segít biztosítani, hogy a szoftver használata kényelmes, érthető és logikus legyen a célközönség számára.

2. Unit tesztelés

- A unit tesztelés fontos szerepet játszik a programkód minőségének fenntartásában. Lehetővé teszi, hogy a fejlesztők még a program összeállása előtt ellenőrizzék az egyes kódrészek működését. Ezzel időben felismerhetők és javíthatók a hibák, ami csökkenti a fejlesztés költségeit és gyorsítja a munkát. A rendszeres unit tesztelés biztosítja, hogy a kód stabil maradjon akkor is, ha később változtatásokat vagy bővítéseket hajtanak végre rajta. Ez különösen hasznos nagyobb projektekben, ahol a megbízhatóság és a hosszú távú karbantarthatóság kulcsfontosságú.

Használt technológiák

A tesztelés során a következő eszközöket használtuk:

- Manuális tesztelés - Chrome
- Unit tesztelés - PHPUnit

Program működésének a részletes leírása

Program részletes leírása

Az online időpontfoglaló rendszer egy modern, felhasználóbarát webalkalmazás, amely lehetővé teszi a látogatók számára, hogy különféle szolgáltatásokra egyszerűen és gyorsan foglaljanak időpontot az interneten keresztül. A rendszer működése a következő főbb lépésekből áll:

1. Bejelentkezés / Regisztráció

A weboldalra látogató felhasználó először a bejelentkező felületre kerül, ahol két lehetőség közül választhat:

- **Regisztráció:** Amennyiben még nincs fiókja, megadhatja az alapvető adatait (pl. név, e-mail cím, jelszó) és regisztrálhat a rendszerbe.
- **Bejelentkezés:** Ha már rendelkezik fiókkal, bejelentkezhet az e-mail címével és jelszavával.

Sikeres bejelentkezés után a felhasználó a főoldalra jut.

2. Főoldal és kategóriaválasztás

A főoldalon a szolgáltatások négy fő kategóriába vannak rendezve, melyek közül a felhasználó szabadon választhat:

- **Ügyintézés**
- **Szépségipar**
- **Oktatás**
- **Egészségügy**

Ezek a kategóriák segítenek a szűrésben, így a felhasználó gyorsabban megtalálhatja a számára releváns szolgáltatást.

3. Szolgáltatás részletes leírása

A kategórián belül megjelennek az elérhető szolgáltatások. Minden egyes szolgáltatáshoz tartozik:

- **A szolgáltatás ára**
- **Egy rövid leírás**, amely tájékoztat a szolgáltatás jellegéről és céljáról

Ez az információ segíti a felhasználót abban, hogy megalapozott döntést hozzon a foglalás előtt.

4. Időpontfoglalás

Amennyiben a felhasználó úgy dönt, hogy igénybe veszi a szolgáltatást, a „**Foglalás**” gombra kattintva elindíthatja az időpontfoglalási folyamatot:

- Kiválasztja a neki megfelelő **időpontot**
- A rendszer rögzíti a foglalást az adatbázisban
- A felhasználó visszajelzést kap a sikeres foglalásról

5. Foglalás kezelése (törlés)

A felhasználók számára biztosított a lehetőség, hogy a már lefoglalt időpontot **később törölhessék**, ha valami közbejön, és mégsem tudnak elmenni. A foglalás törlése után a rendszer felszabadítja az adott időpontot, így az más számára ismét elérhetővé válik.

Fejlesztői dokumentáció

Fejlesztési technológiák

- Frontend:
 - A felhasználói felület megvalósításához **JavaScript** nyelvet alkalmaztunk, amely lehetővé teszi az interaktív elemek és dinamikus tartalmak kezelését a böngésző oldalon. A JavaScript segítségével valósítottuk meg például a menük dinamikus működését, az űrlapellenőrzést, valamint az AJAX-alapú aszinkron adatkommunikációt.
 - A megjelenés kialakításához **Bootstrap** keretrendszert is használtunk, amely biztosítja a reszponzív dizájnt és a modern, egységes megjelenést különböző eszközökön. A Bootstrap segítségével gyorsan és hatékonyan tudtunk formázott elemeket, például gombokat, kártyákat, navigációs sávokat létrehozni.
- Backend:
 - A szerveroldali logika kialakításához **PHP** nyelvet használtunk. A PHP gondoskodik az adatok feldolgozásáról, az adatbázissal való kommunikációról, valamint a különféle backend funkciók, például a felhasználói regisztráció, bejelentkezés és időpontfoglalás kezeléséről.
- Adatbázis:
 - Az alkalmazás adatainak tárolására **MySQL** adatbázist használtunk. A MySQL megbízható és széles körben használt relációs adatbáziskezelő rendszer, amely jól illeszkedik a PHP-vel való integrációhoz. Az adatbázisban tároljuk például a felhasználói adatokat, az időpontokat, valamint az adminisztrációhoz szükséges egyéb információkat.

HTTP

protokollok

A szerver-kliens kommunikáció során a kérések között két alapvető HTTP módszert alkalmaztunk:

- **GET:** A GET kéréseket olyan esetekben használtuk, amikor adatokat kellett lekérni a szervertől anélkül, hogy módosítottuk volna azokat. Például a felhasználói profilok vagy az elérhető időpontok megjelenítéséhez GET kérések kerültek kiküldetésre. A GET módszer URL-en keresztül továbbítja a paramétereket, így elsősorban nem érzékeny adatok lekérdezésére alkalmazható.

- **POST:** A POST metódust akkor használtuk, amikor adatokat kellett elküldeni a szervernek, például űrlapok beküldései. Ilyen eset volt például a regisztráció, bejelentkezés, vagy egy új időpont lefoglalása. A POST kérés lehetővé teszi az adatok biztonságosabb és strukturáltabb továbbítását, mivel azok nem az URLben, hanem a kérés törzsében kerülnek elküldésre.

A rétegek szerepköre a projektben

A projekt a **rétegzett architektúra** elvét követi, amely elősegíti a kód jobb átláthatóságát, karbantartását és újrafelhasználhatóságát. Az alábbiakban ismertetjük az egyes rétegek szerepét, különös tekintettel a vezérlő (Controller) rétegre:

1. Controller réteg

Ez a réteg felelős a felhasználói kérések kezeléséért, az adatok előkészítéséért és a nézetek kiszolgálásáért. A controllerek összekötik a felhasználói felületet (frontend) és az üzleti logikát (modellek/adatbázis).

A projektben az alábbi controller osztályokat hoztuk létre:

- **AppointmentController.php** – Kezeli az időpontfoglalással kapcsolatos műveleteket, például új időpont létrehozása, törlése vagy listázása.
- **mainpageController.php** – A főoldalhoz tartozó vezérlő, amely az alapvető oldalelemeket szolgálja ki.
- **ProviderController.php** – A szolgáltatók (pl. fodrászok, orvosok stb.) adatainak kezelését végzi.
- **ServiceController.php** – A különböző szolgáltatásokhoz kapcsolódó logikát tartalmazza.
- **UserController.php** – A felhasználók regisztrációját, bejelentkezését, adatainak módosítását kezeli.

2. Model réteg

A modellek felelnek az adatkezelésért és az adatbázissal való kommunikációért. Itt történik az adatok lekérdezése, módosítása és validálása.

3. View réteg

Ez a réteg felelős a megjelenítésért, vagyis az adatok felhasználóbarát formában való bemutatásáért. A nézetek HTML, CSS (és Bootstrap) segítségével valósulnak meg.

Frontend részletes leírása

Felhasználói azonosítás

Az oldal betöltésekor először ellenőrzi, hogy a felhasználó be van-e jelentkezve. Ha nem, automatikusan visszairányítja az index oldalra.

Adatlekérés

Az oldal a ratings tábla alapján lekéri az összes szolgáltatóhoz tartozó értékelések átlagát, amit később a szolgáltatók kártyáin meg is jelenít.

Felület és navigáció • Modern, reszponzív design sötét/világos mód

támogatással.

- Egy lebegő kis menü segítségével váltható a sötét mód, ahol egy valós idejű óra is látható.
- A felső navigációs sávban elérhetőek a fő menüpontok:
Főoldal, Felfedezés, Időpontjaim, Saját fiók, valamint megjelenik a bejelentkezett felhasználó neve.
- Beépített kereső segíti a gyors navigációt.

Szolgáltatások szűrése és megjelenítése

A szolgáltatásokat típus szerint lehet szűrni (pl. **Egészségügy, Szépségipar, Oktatás, Adminisztratív**).

A szűrés JavaScript segítségével történik, és dinamikusan frissíti a megjelenített szolgáltatásokat.

Minden szolgáltatóhoz tartozik egy **kártya**, amely tartalmazza:

- a nevét,
- képét (ha nincs feltöltve, helyettesítő képet használ),
- az átlagos értékelését csillag ikonokkal.

Szolgáltatás hozzáadása

Ha a bejelentkezett felhasználó **nem "customer"** szerepkörű, akkor megjelenik egy **„Szolgáltatás hozzáadása”** gomb. • A gombra kattintva egy modális ablak nyílik meg egy űrlappal.

- Az űrlap mezői: szolgáltatás típusa, neve, leírása, nyitvatartás (validált formában), cím, ár, időtartam, kép feltöltése.
- A rendszer ellenőrzi az adatokat, és hiba esetén figyelmeztető üzenetet ad vissza.

Szolgáltató profil megtekintése

- A felhasználó rákattinthat egy szolgáltató kártyájára, ami megnyit egy részletes **modális ablakot**.
- A modálban megjelennek a szolgáltató adatai, képei, és elérhető időpontjai.
- Innen közvetlenül lehet **időpontot foglalni**, kivéve, ha a felhasználó **admin**.

Lábléc

A weboldal alján található egy letisztult **lábléc**, amely tartalmazza:

- a Bookify logóját,
- elérhetőségi adatokat (e-mail, telefonszám, cím), • hasznos linkeket (pl. GYIK, Kapcsolat),
- közösségi média ikonokat.

JavaScript működés

Az oldal működését több JS fájl támogatja:

- `ratings.js` – értékelések megjelenítése
- `booking.js` – időpontfoglalás kezelése
- `account.js` – profilkezelés
- `theme.js` – sötét/világos mód vezérlése
- `appointments.js` – időpontok kezelése
- `filter.js` – szolgáltatás szűrés
- `service.js` – szolgáltató profil betöltés
- `time.js` – óra megjelenítése

Frontend egyes részein kiemelkedő megoldások

Ez a JavaScript kód egy időpontfoglaló rendszer működését vezérli. Amikor az oldal betöltődik, a rendszer megpróbálja beazonosítani a felhasználót, hogy tudja, ki próbál időpontot foglalni. Ehhez először megnézi, hogy van-e egy globális `userId` változó, ha nincs, akkor a HTML dokumentumból (a `<body>` elem adataiból) próbálja kiolvasni.

Ezután beállítja, hogy amikor a felhasználó rákattint egy szolgáltató kártyájára, akkor megnyíljon egy modal ablak (egy felugró ablak), ahol részletes információkat láthat a szolgáltatóról, és ott tud majd időpontot is foglalni.

A modal ablakon belül, ha a felhasználó rákattint a "Foglalás" gombra, akkor megjelenik egy naptár, ahol ki tudja választani a számára megfelelő napot és időpontot. A rendszer lekéri az adott szolgáltató nyitvatartási idejét, és csak azokra a napokra enged foglalást, amikor tényleg elérhető a szolgáltató.

A naptár figyeli, hogy mely időpontok már foglaltak, ezeket pirossal jelzi. A felhasználó csak szabad, érvényes időpontokra tud kattintani. Ha kiválasztott egy időpontot, akkor a rendszer ezt megjegyzi, és ha a felhasználó megerősíti a foglalást, elküldi az adatokat a szervernek, amely elmenti az időpontfoglalást.

Ha a modal ablakot bezárják, a naptár törlődik, és az ablak visszaáll az alapállapotba. Így minden új foglalásnál egy tiszta nézet fogadja a felhasználót.

Szolgáltatás részletei

< >

Ma

2025. Április

Hónap

Hétfő	Kedd	Szerda	Csütörtök	Péntek	Szombat	Vasárnap
31	1	2	3	4	5	6
7	8	9	10	11	12	13
14	15	16	17	18	19	20
21	22	23	24	25	26	27
28	29	30	1	2	3	4
5	6	7	8	9	10	11

Foglalás

Lokális szerver futtatása

A fejlesztés során a Bookify alkalmazást **lokális szerveren** futtattuk a **MAMP** (Macintosh, Apache, MySQL, PHP) segítségével. Ez egy könnyen használható eszköz, amely lehetővé teszi, hogy a teljes webalkalmazást teszteljük a saját gépen, internetkapcsolat nélkül.

Csapatmunka

Szerepek

- **Husi Kristóf – Projektmenedzser & Backend fejlesztő**
Kristóf irányította a projekt menetét, ő koordinálta a csoportmunkát, figyelte a határidőket, és biztosította a feladatok egyenletes elosztását. Emellett a **backend fejlesztéséért** felelt, ő készítette el a PHP alapú szerveroldali logikát, a foglalási rendszer működését, valamint az adatbáziskapcsolatot.
- **Erdei Tamás – Frontend fejlesztő**
Tamás feladata volt a felhasználói felület kialakítása. A Bootstrap keretrendszer segítségével letisztult, responszív dizájnt valósított meg, külön figyelmet fordítva a könnyű kezelhetőségre és az átláthatóságra.
- **Bräutigam Richárd – Adatbázis-felelős**
Richárd tervezte meg és valósította meg az adatbázist. Ő készítette el az adatbázis-struktúrát, a relációkat, valamint az SQL szkripteket az importáláshoz. Az adatbázis megfelelő kialakítása kulcsszerepet játszott a rendszer hatékony működésében.

Jövőbeli terveink

A projekt jövőbeli tervei a következő kulcsfontosságú fejlesztéseket tartalmazzák, amelyek jelentősen hozzájárulnak a rendszer bővítéséhez és a felhasználói élmény javításához:

1. **Online fizetés:** Az online fizetési rendszer integrálása lehetővé teszi a felhasználók számára, hogy kényelmesen és biztonságosan végezzenek tranzakciókat az alkalmazásban. Ezzel a funkcióval az ügyfelek gyorsan és egyszerűen fizethetnek az időpontfoglalásokért vagy szolgáltatásokért, anélkül hogy el kellene hagyniuk az alkalmazást. A legelterjedtebb online fizetési rendszerek, mint például PayPal, Stripe vagy helyi banki megoldások beépítése várható.
2. **Hirdetéstámogatás:** A hirdetési lehetőségek beépítése segítheti a projekt monetizálását, miközben a felhasználóknak releváns és érdekes hirdetéseket kínálhat. A hirdetések célzottak lehetnek, például olyan szolgáltatások vagy termékek reklámozására, amelyek kapcsolódnak az alkalmazásban foglalt időpontokhoz vagy egyéb releváns területekhez.
3. **Terjessze ki külföldre:** A nemzetközi piacra való belépés új lehetőségeket biztosít a rendszer számára. A projekt többnyelvű támogatása és a különböző országok jogi és

pénzügyi követelményeihez való igazodás fontos lépései ennek a bővítésnek. Külföldi partnerek és felhasználók bevonása a szolgáltatás népszerűsítésére, a piacon való növekvő jelenlét biztosítására és az alkalmazás globális elterjedésére összpontosít.

4. **Ügyfélszolgálat:** A felhasználói élmény javítása érdekében ügyfélszolgálatot indítunk, amely gyors és hatékony segítséget biztosít a felmerülő problémák vagy kérdések megoldásában. Az online chat, emailés vagy telefonos támogatás lehetőségei mind hozzájárulnak a felhasználók elégedettségéhez és a szolgáltatás minőségének javításához. Az ügyfélszolgálat automatizált chatbotokkal is kiegészíthető, amelyek gyorsan válaszolnak a leggyakoribb kérdésekre.
5. **Mobil App:** A mobilalkalmazás fejlesztése lehetővé teszi a felhasználók számára, hogy bárhol és bármikor könnyedén intézhessék az időpontfoglalásokat és kezelhessék a szolgáltatásokat. A mobil alkalmazás felhasználóbarát felülete egyszerűsíti a navigációt, és biztosítja, hogy az ügyfelek gyorsan hozzáférjenek minden szükséges információhoz. A push értesítések révén a felhasználók valós idejű információkat kaphatnak az időpontjaikkal kapcsolatban.